

EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA

TECNOLOGICO NACIONAL DE MEXICO

INSTITUTO TECNOLÓGICO DE CIUDAD MADERO

TECNOLÓGICO NACIONAL DE MÉXICO

Bot de telegram

Nelson Hernández Morales

POR MI PATRIA Y POR MI BIEN

Índice

1. El bot
2. El taller: el ejemplo que veremos
3. El bot y los mensajes
4. Procesos externos
5. Archivos de Telegram
6. Ejecución y práctica

Un bot que ya está en operación

- Es el bot `@Framework_Visthaa_bot`.
- Apoya a un tomador de decisiones (DM) a elegir un plan de remodelación.
- El DM no programa: observa soluciones y pulsa botones.
- Cada solución describe 9 áreas de la casa y 5 costos.
- El mismo conjunto de datos viaja en JSON, CSV o XML.

Las 9 áreas

Baño, cocina, recámara, cochera, sala, jardín, sótano, ático y recibidor.

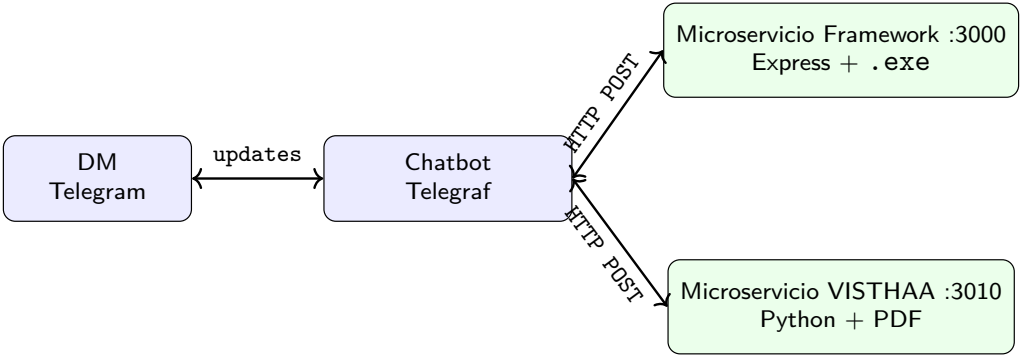
Los 5 costos

Material, mano de obra, tiempo, energía eléctrica y agua.

La idea central

El bot no calcula la optimización: coordina programas que ya existen y convierte sus resultados en una conversación.

Tres procesos que colaboran



- Solo el Chatbot habla con Telegram.
- Framework y VISTHAA son microservicios locales.
- Los servicios no comparten memoria.
- El Chatbot conserva el estado de la conversación.

La conexión entre microservicios

Microservicio	Puerto	Responsabilidad
Chatbot	—	Recibe Telegram y orquesta el flujo
Framework	3000	Ejecuta los `.exe` y responde la matriz
VISTHAA	3010	Ejecuta Python y devuelve el PDF

- El Chatbot llama a `http://localhost:3000` y `http://localhost:3010`.
- Cada petición lleva los datos que el servicio necesita.
- El Framework usa Express; VISTHAA usa Express y Python con matplotlib.

Lo importante

Los microservicios no se conocen entre sí: el Chatbot es quien los une y decide en qué orden llamarlos.

Dentro del Framework: los ejecutables como cajas negras

- Express recibe y decodifica la matriz en JSON, XML o CSV.
- El Framework arma los argumentos con el ID del usuario y la iteración.
- Ejecuta `Compromiso.exe`, `_MOEAD.exe` o `DOE_PDA.exe` y espera su salida.
- Los `.exe` escriben texto en `resultados/`; el Framework lo lee y lo convierte en matriz.

```
let child = shell.exec(
  comandoExe('./MSDOSS/_MOEAD.exe',
    `${user_id} ${iterador} ${100}`),
  { async: true }
);
```

El contrato de la caja negra

No se modifica el binario: se documenta qué recibe, qué archivos produce y cómo se lee su salida.

Una petición concreta

`POST http://localhost:3000/main/JSON`

Parte	Ejemplo	Para qué sirve
Ruta	`/main/XML`	Elige servicio, etapa y formato
Cuerpo	`user_id, iterador`	Lleva los datos de la sesión
Content-Type	`application/json`	Indica cómo leer el cuerpo

```
axios.post(BASE_FRAMEWORK + '/main/' + formato,
  data, config);
```

El recorrido de una sesión

#	Acción del DM	Qué ocurre en el sistema
1	`/json`, `/xml` o `/csv`	`POST /aleatorias/...`: 100 soluciones; se muestran 10.
2	Pulsa `Sol. N`	Se guarda el compromiso y se llama a `/main/...`.
3	Recibe una solución a la vez	`_MOEAD.exe` produce el conjunto MOEA/D; se evalúan 10.
4	Pulsa `Si` o `No`	Se guarda 1 o 0 en el vector de referencia.
5	Repite la evaluación	`POST /pda/...`: se ejecutan PDA y MOEA/D.
6	Termina la tercera iteración	Se muestra la mejor solución y se ofrece la gráfica una sola vez.

La gráfica final se pide a VISTHAA y se entrega como PDF por Telegram.

Los botones son callback queries

Cuando el bot envía un teclado en línea, cada botón lleva un identificador interno llamado `callback_data`. Al pulsarlo, Telegram no envía un mensaje de texto: envía un **callback query**.

Botón	callback_data	Qué significa
`Sol. 1` … `Sol. 10`	`S1` … `S10`	Elige la solución de compromiso
`Si` / `No`	`SI1` / `NO1`	Califica una solución: 1 o 0
`Si` / `No` (gráfica)	`SIV` / `NOV`	Genera o no el PDF final

- Telegraf recibe el callback query y lo entrega a `bot.action(...)`.
- `ctx.match[0]` contiene el `callback_data` que se pulsó.
- `ctx.update.callback_query.message` identifica el mensaje y el chat.
- El `callback_data` es un identificador corto (máximo 64 bytes), no un comando libre.

Qué ocurre al pulsar un botón

```
bot.action(['SI1', 'NO1'], async (ctx) => {
  const q = ctx.update.callback_query;

  // Quita los botones para que no se pulse dos veces.
  await bot.telegram.editMessageReplyMarkup(
    q.message.chat.id,
    q.message.message_id,
    undefined,
    { reply_markup: {} }
  );

  // 1 = SI1, 0 = NO1; despues continua el ciclo.
});
```

- El bot guarda el mensaje editado en `mensajesEditados`.
- Si llega otra vez el mismo botón, se ignora.
- El teclado se retira después de usarlo.
- `ctx.answerCbQuery()` es buena práctica para detener el indicador de carga.

El estado de la sesión vive en memoria

Variable	Qué guarda
`solucionesAleatorias`	Las soluciones iniciales; se muestran 10
`solucionesMOEADO`	El conjunto MOEA/D de la iteración actual
`soluciones_buenas_malas`	Un 1 o 0 por cada solución calificada
`veces_respondido`	Cuántas soluciones se han calificado en la ronda
`iterador_principal`	La iteración del PDA (0 a 3)

- `/json`, `/xml` o `/csv` reinician el estado antes de empezar.
- Si el bot se reinicia, la conversación se pierde: hay que empezar de nuevo.

El bot de maestría

- Tres procesos y dos puertos.
- Tres `.exe` de consola.
- Python con matplotlib para el PDF.
- JSON, CSV y XML.
- Estado global en memoria.

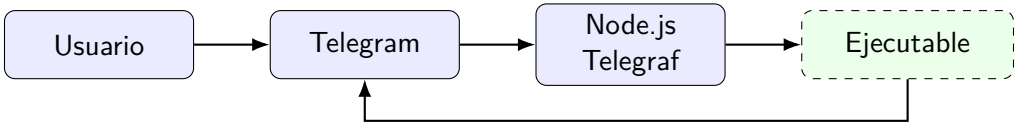
El taller

- Un solo proceso Node.js.
- Un `.exe` pequeño.
- Un script de Python.
- Un archivo PDF y una imagen.



El patrón es el mismo; el taller lo muestra en pequeño y con todo el código a la vista.

Flujo del bot de Telegram



La tarea del bot

Recibir un evento, ejecutar la acción adecuada y responder en el chat.

Qué hace Telegraf

- Conecta la aplicación Node.js con la API de Telegram.
- Entrega cada actualización a un middleware y luego al handler correspondiente.
- `ctx` contiene el evento y permite responder al chat.
- `bot.launch()` inicia el long polling que usa este ejemplo.

Ciclo básico

Telegram → update → middleware → handler → respuesta

Un solo proceso debe hacer polling con cada token.

El objeto ctx

¿Qué es el objeto ctx?

El `ctx` (Contexto) es un paquete temporal que contiene los **datos** del evento recibido (quién, qué, dónde) y las **herramientas** directas para responder (ej. `ctx.reply()`).

```
bot.use((ctx, next) => {
        // Registrar el tipo, no el objeto ctx ni el texto del usuario.
        const mensaje = ctx.message;
        const tipo = mensaje?.text ? 'texto'
        : mensaje?.photo ? 'foto'
        : mensaje?.document ? 'documento'
        : ctx.updateType;

        console.log(`[update] ${ctx.updateType}: ${tipo}`);
        return next();
});
```

Comandos que tiene este bot

Comando	Acción
`/sumar 2 3`	Suma con el script de Python
`/multiplicar 4 2.5`	Multiplica con Python
`/exe sumar 7 5`	Ejecuta la calculadora `.exe`
`/calcular 7 5`	Muestra botones para elegir
`/pdf`	Envía el PDF local de ejemplo
`/imagen`	Envía la imagen local de ejemplo
`/id`	Muestra tu id de Telegram
`/start`, `/help`	Muestra la ayuda

Los handlers de documento y foto también aceptan archivos enviados como mensajes, no como comandos.

Handler: validar, ejecutar y responder

```
bot.command('sumar', async (ctx) => {
        const numeros = leerDosNumeros(ctx.payload);
        if (!numeros) {
                return responder(ctx, 'Uso: /sumar 2 3');
        }

        try {
                const { stdout } = await proc.ejecutarPython(
                'sumar', numeros[0], numeros[1]
                );
                await responder(ctx, stdout);
        } catch (error) {
                await responder(ctx, textoError(error));
        }
});
```

Este ejemplo refleja el handler del proyecto: comprueba los datos antes de crear el proceso.

Botones: teclado en línea y callback query

1. El bot envía un mensaje con un **teclado en línea**.
2. Cada botón lleva un `callback_data` (máximo 64 bytes).
3. Al pulsarlo, Telegram no manda texto: manda un **callback query**.
4. Telegraf lo entrega a `bot.action(...)`.

En el taller

`/calcular 7 5` muestra cuatro botones: sumar o multiplicar, con Python o con el `.exe`.

Crear el teclado

```
const boton = (texto, operacion, motor) =>
        Markup.button.callback(
                texto,
                `calc:${operacion}:${motor}:${a}:${b}`
        );

const teclado = Markup.inlineKeyboard([
        [boton('Sumar con Python', 'sumar', 'py'),
         boton('Sumar con .exe', 'sumar', 'exe')],
]);

await ctx.reply('Que hago con 7 y 5?', teclado);
```

- Cada lista interior es una fila de botones.
- El `callback_data` lleva todo lo necesario: el bot no guarda estado entre el comando y el clic.

Atender el clic

```
bot.action(PATRON, async (ctx) => {
        const [, operacion, motor, a, b] = ctx.match;

        await ctx.answerCbQuery('Calculando...');
        await ctx.editMessageText('Ejecutando...');

        try {
                const { stdout } = await ejecutar(operacion, a, b);
                await ctx.editMessageText(stdout);
        } catch (error) {
                await ctx.editMessageText(textoError(error));
        }
});
```

- `PATRON` es una expresión regular, por ejemplo `/^calc:(sumar|multiplicar):.../`.

Qué hace cada llamada

- `ctx.match`: lo que capturó la expresión regular del `callback_data`.
- `ctx.answerCbQuery()`: contesta el clic y apaga el relojito del botón.
- `ctx.editMessageText()`: cambia el mensaje y, si no se indica `reply_markup`, quita los botones.

Un botón también es entrada del usuario

El `callback_data` no es de fiar: `lib/procesos.js` vuelve a validar la operación y los números, igual que con un comando.

Qué hace `lib/procesos.js`

- Acepta solo dos operaciones: `sumar` y `multiplicar`.
- Comprueba que ambos operandos sean números finitos.
- Usa `execFile(programa, args)`: el programa y sus argumentos van separados.
- Fija la carpeta de trabajo, oculta la ventana y limita tiempo y salida.

Límites configurados

Tiempo máximo: 5 segundos. Salida estándar y de error: 100 KB.

La llamada externa, sin concatenar comandos

```
execFile(programa, args, {
        cwd: RAIZ,
        timeout: 5000,
        maxBuffer: 100 * 1024,
        windowsHide: true,
        encoding: 'utf8',
}, (error, stdout, stderr) => {
        // error indica que el programa fallo.
});
```

- El proceso hijo usa la cuenta de Windows que ejecuta el bot.
- No debes pasar texto del usuario a una cadena de `exec`.
- La calculadora del taller es un ejemplo; el ejecutable de la tesis se mantiene como caja negra.

Recibir y guardar un archivo

1. Telegram entrega un `file_id` y metadatos.
2. `getFileLink()` obtiene un enlace temporal.
3. `fetch()` lee la respuesta por partes.
4. El bot guarda el archivo con nombre saneado y un prefijo único.

Protecciones del ejemplo

Límite de 10 MB, comprobación del tamaño informado y real, limpieza del nombre y cálculo de SHA-256. El archivo recibido nunca se ejecuta.

Enviar el PDF y la imagen de ejemplo

- `/pdf` lee `datos/paraEnviar/ejemplo.pdf`.
- `/imagen` lee `datos/paraEnviar/ejemplo.jpg`.
- Axios envía el archivo como formulario multipart a la API de Telegram.
- Si falla el envío, el bot hace hasta tres intentos, con dos segundos de pausa entre ellos.

Una diferencia importante

Descargar un archivo es recibir datos desde Telegram; enviar uno es subir un archivo local a Telegram.

Token, permisos y cuidado de datos

- El token se guarda en `.env`, no en el código ni en los registros.
- Los errores de red de Telegraf incluyen el token: el bot lo oculta antes de imprimir.
- `USUARIOS_PERMITIDOS` limita quién usa el bot, incluidos sus botones.
- Ejecuta el bot con una cuenta normal, no como administrador.
- Valida comandos, números, nombres y tamaño de los archivos.
- Conserva el control de errores y los tiempos máximos de espera.

Estructura del taller y prueba guiada

- `index.js`: Telegraf, middleware y handlers.
- `lib/procesos.js`: validación y llamadas a procesos.
- `lib/archivos.js`: descarga y seguridad de archivos.
- `bin/` y `python/`: programas de ejemplo.
- `datos/paraEnviar/`: archivos que envían los comandos.

Prueba

1. Configura `BOT_TOKEN`.
2. Ejecuta `npm install`.
3. Arranca con `npm start`.
4. Prueba `/calcular 7 5` y pulsa un botón.
5. Envía un archivo menor a 10 MB.

Para recordar

1. Telegraf convierte actualizaciones de Telegram en trabajo para los handlers.
2. El handler valida antes de llamar a Python o al `.exe`.
3. `execFile` recibe argumentos separados y tiene límites de ejecución.
4. El bot distingue entre descargar un archivo y enviar uno local.
5. Un botón envía un callback query: se contesta, se edita el mensaje y se valida de nuevo.

¿Preguntas?